



DEEP LEARNING

A Beginner-Friendly Story

- Start with a simple prediction
- See how a neuron thinks
- Measure mistakes with loss
- Learn through backpropagation
- Improve with an optimizer

Prepared for Somnath Singh

22-page structured reference | Definitions, examples, links, and revision maps

1. The Whole Story in One Page

Understand the complete journey before learning individual terms

Imagine teaching a computer to predict whether a student will pass an exam. You give the computer examples: study hours, sleep hours, previous marks, and the final result. Deep learning is the story of how the computer changes those examples into a useful prediction.



One-sentence definition

Deep learning is a method in which a multi-layer neural network learns useful weights from many examples by repeatedly predicting, measuring error, and correcting itself.

2. Beginner Vocabulary

The words used throughout the story

Word	Beginner meaning	Exam example
Input / feature	Information given to the model	Study hours
Target / label	Correct answer to learn	Pass = 1, fail = 0
Weight	Importance of an input	Study hours may receive more weight
Bias	Extra adjustable starting value	Shifts the decision boundary
Neuron	Small calculation unit	Combines weighted inputs
Layer	Group of neurons	Hidden layer
Model	The complete learned system	Student pass predictor
Prediction	Model answer	0.82 chance of pass
Parameter	A learned weight or bias	w1, w2, b
Hyperparameter	A setting chosen before training	Learning rate, batch size

Say it simply

A model contains layers. Layers contain neurons. Neurons use weights and biases. Training changes those weights and biases.

3. Where Deep Learning Fits

AI -> machine learning -> deep learning

Three related ideas

Artificial intelligence is the broad idea of making computers perform useful tasks that appear intelligent. Machine learning is one way to do this by learning patterns from data. Deep learning is a type of machine learning that uses neural networks with multiple layers.

Artificial Intelligence

Machine Learning

Deep Learning

Why “deep”?

- Many transformation layers
- Each layer learns a different pattern
- Later layers build on earlier layers

Network families and the data they understand

Family	Think of it as	Common data
ANN / MLP	General pattern learner	Tables and numeric features
CNN	Looks for spatial patterns	Images and video frames
RNN / LSTM / GRU	Remembers a sequence	Text and time series
Transformer	Connects important parts of a sequence	Language, vision, multimodal data

Connection

This guide first explains the common learning process shared by neural networks. Specialized networks change the internal architecture, but prediction, loss, backpropagation, and optimization remain central ideas.

4. Start with Data

Features, labels, examples, and preparation

Story step 1: Collect examples

A neural network does not begin with knowledge. It begins with examples. One row is one example. The input columns are features, and the correct answer is the label.

How this connects

After data is prepared, the model needs a mathematical unit that can combine the features. That unit is the neuron.

Example dataset

Study hours	Sleep hours	Previous marks	Result
2	5	42	Fail (0)
5	7	68	Pass (1)
8	7	84	Pass (1)

Why preparation matters

- Numeric features may use very different scales. Scaling prevents one large-number feature from dominating only because of its unit.
- Missing or incorrect values can teach the model the wrong pattern.
- Training data teaches the model. Validation data helps choose settings. Test data checks the final model on unseen examples.

Avoid data leakage

Do not let information from validation or test data influence training. Leakage can make reported performance look better than real future performance.

5. The Artificial Neuron

The smallest decision-making calculation

Story step 2: Combine the inputs

A neuron receives numbers, multiplies each number by a weight, adds a bias, and then sends the result through an activation function.

How this connects

The weighted sum tells us the strength of the signal. The activation function decides how the signal should continue to the next layer.

WEIGHTED SUM

$$z = w_1 * x_1 + w_2 * x_2 + \dots + w_n * x_n + b$$

NEURON RESULT

$$\text{output} = \text{activation}(z)$$

Meaning of every symbol

Symbol	Meaning
x	Input or feature
w	Weight: learned importance of that input
b	Bias: adjustable offset
z	Weighted sum before activation
output	Signal sent to the next neuron or final answer

Simple analogy

Think of a hiring scorecard. Each criterion has importance, the scores are combined, and a rule converts the total into a decision. A neuron performs a similar mathematical process.

6. A Neuron Example with Numbers

See exactly how one prediction signal is created

Suppose the model sees this student

- Study hours $x_1 = 6$
- Sleep hours $x_2 = 7$
- Weight for study $w_1 = 0.8$
- Weight for sleep $w_2 = 0.3$
- Bias $b = -5$

STEP 1: WEIGHTED SUM

$$z = (0.8 * 6) + (0.3 * 7) - 5 = 1.9$$

The value 1.9 is not yet a probability. It is a raw score, often called a logit when it is produced before a classification output function.

What the weights tell us

- The study weight is larger than the sleep weight in this example, so one extra study hour changes the score more.
- A positive weight pushes the score upward when the input increases. A negative weight pushes the score downward.
- The bias shifts the whole decision without depending on a particular feature.

Important

These weight values are only a teaching example. During real training, the model learns suitable values from data.

Next connection

The raw score must now be transformed into a useful signal. That is the job of an activation function.

7. From One Neuron to a Neural Network

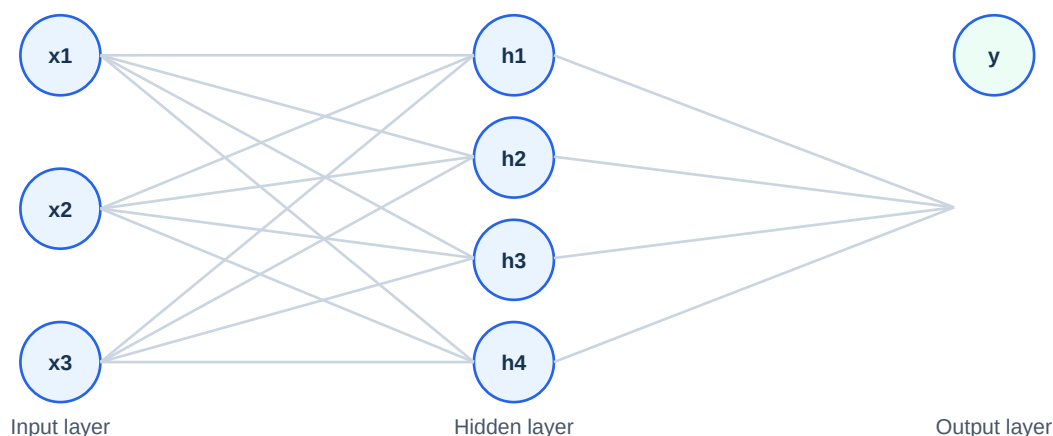
Perceptron, layers, depth, and nonlinear learning

Perceptron

A perceptron is a basic artificial neuron used for a simple decision. A single perceptron creates a linear boundary, such as a straight line separating two classes. It cannot solve every curved or complex pattern.

Multi-layer perceptron or MLP

An MLP connects many neurons in layers. The input layer receives features. Hidden layers learn intermediate patterns. The output layer gives the final prediction.



- Depth means the number of transformation layers.
- Width means the number of neurons in a layer.
- Nonlinear activation functions allow several layers to learn curved and complex relationships.

Key connection

Without nonlinear activation functions, many linear layers still behave like one linear transformation. Activation functions give the network the ability to learn richer patterns.

8. Forward Propagation

The prediction moves from left to right

Story step 3: Make a prediction

Forward propagation means sending the input through the network one layer at a time until an output is produced.

How this connects

Once the model has produced an answer, the answer must be compared with the correct label. That comparison is the loss.

What happens in each layer?

- Receive outputs from the previous layer.
- Calculate weighted sums using weights and biases.
- Apply an activation function.
- Send the new outputs to the next layer.

HIDDEN LAYER

$$z[1] = W[1]x + b[1] \quad a[1] = f(z[1])$$

OUTPUT LAYER

$$z[2] = W[2]a[1] + b[2] \quad \text{prediction} = g(z[2])$$

Plain-English translation

The first layer changes raw features into learned signals. The next layer combines those signals. The final layer changes them into the answer required by the task.

Remember

Forward propagation does not learn by itself. It only uses the current weights to calculate the current prediction.

9. Why Activation Functions Are Needed

Turning a raw score into a useful signal

Definition

An activation function is a rule applied to a neuron's weighted sum. It decides the neuron output and introduces nonlinearity, which allows the network to learn relationships that are not simple straight lines.

Activation functions have two main jobs

- Hidden layers: help the network build complex patterns.
- Output layer: shape the answer for the task, such as a probability or a numeric value.

Output activation selection

Task	What the answer means	Output activation
Regression	Any suitable number	Linear
Binary classification	Probability of one of two classes	Sigmoid
Single-label multiclass	Probabilities that sum to 1	Softmax
Multi-label classification	Separate probability for each label	Sigmoid per label

Story connection

The hidden activation helps create useful internal signals. The output activation gives the prediction the correct form. The loss function must agree with that output form.

10. Sigmoid and Tanh

Two smooth, bounded activation functions

Sigmoid

IN SYMBOLS

$$\text{sigmoid}(z) = 1 / (1 + \exp(-z))$$

- Output range: between 0 and 1.
- Easy to read as a binary probability.
- Large positive or negative inputs enter flat regions where gradients become very small.
- Sigmoid outputs are not centered around zero.

Tanh

IN SYMBOLS

$$\tanh(z) = (\exp(z) - \exp(-z)) / (\exp(z) + \exp(-z))$$

- Output range: between -1 and 1.
- Outputs are centered around zero.
- Like sigmoid, tanh can saturate and produce small gradients.

When to use

Function	Good beginner use
Sigmoid	Final output for binary classification
Tanh	A bounded hidden state when negative and positive signals are useful

Saturation

Saturation means the curve becomes almost flat. When the curve is flat, changing the input causes very little output change, so the gradient becomes small.

11. ReLU and Its Variants

Fast hidden-layer activations and the dead-neuron issue

ReLU

IN SYMBOLS

$$\text{ReLU}(z) = \max(0, z)$$

- Negative input becomes 0. Positive input passes through unchanged.
- Calculation is simple and fast.
- The positive side has a useful gradient, so ReLU often trains deep networks better than sigmoid or tanh.

Dead ReLU problem

If a neuron keeps receiving negative weighted sums, ReLU keeps returning zero. Its gradient is then zero on that side, so the neuron may stop updating. This is called a dead neuron.

Variants

Function	Negative input behavior	Why it helps
Leaky ReLU	Small fixed negative slope	Keeps a small gradient
PReLU	Learns the negative slope	Lets the model choose the slope
ELU	Smooth negative curve	Allows negative output; smoother transition

Beginner default

For a basic feed-forward network, ReLU is a practical hidden-layer starting point. If many neurons die, try Leaky ReLU or review learning rate and initialization.

12. Softmax

Turning several class scores into class probabilities

A three-class story

Suppose an image model must choose one class: cat, dog, or horse. The final layer produces three raw scores. Softmax converts those scores into three probabilities that add up to 1.

IN SYMBOLS

$$\text{softmax}(z_i) = \exp(z_i) / \sum(\exp(z_j))$$

Class	Raw score	Softmax probability
Cat	1.2	Example probability
Dog	2.8	Highest probability
Horse	0.4	Example probability

- A larger raw score receives a larger probability.
- The probabilities are linked because increasing one affects the others.
- Softmax is suitable when exactly one class should be correct.

Do not confuse

For multi-label tasks, more than one label may be true at the same time. Use separate sigmoid outputs rather than one softmax distribution.

Next connection

Now the model has a prediction. We need a number that tells us how wrong that prediction is. That number is the loss.

13. Loss Functions

A score for how wrong the prediction is

Story step 4: Measure the mistake

A loss function compares the prediction with the correct answer. A small loss means the prediction is close to the target. A large loss means the prediction is far from the target.

How this connects

Backpropagation uses the loss to find how each weight contributed to the mistake.

Loss, batch loss, and cost

Term	Simple meaning
Loss	Error for one example
Batch loss	Average or total error for one batch
Cost / objective	Overall quantity minimized during training

Why not only count correct answers?

An accuracy count says only correct or incorrect. A loss also tells how confident and how far wrong the model was. This detailed signal helps the optimizer decide how to change the weights.

Example

Predicting 0.49 for a true class of 1 and predicting 0.01 for the same class are both incorrect at a 0.5 threshold, but the second prediction is much more wrong. A suitable loss reflects that difference.

14. Regression Losses

When the answer is a number

Example task: predict a house price

The target and prediction are numeric values. Regression losses measure the distance between them.

Loss	Beginner definition	Main behavior
MSE	Average squared error	Large errors receive a much bigger penalty
MAE	Average absolute error	More resistant to unusual large errors
Huber	MSE for small errors, MAE-like for large errors	Balances smooth learning and robustness
RMSE	Square root of MSE	Reported in the same unit as the target

Small example

IN SYMBOLS

target = 100, prediction = 90, error = 10

- Absolute error is 10.
- Squared error is 100.
- Squaring makes larger mistakes grow quickly.

Selection idea

Choose MSE when large errors should be strongly punished, MAE when outliers should have less influence, and Huber when you want a balance.

15. Classification Losses

When the answer is a category

Binary cross-entropy

IN SYMBOLS

$$L = -[y \cdot \log(p) + (1-y) \cdot \log(1-p)]$$

Use binary cross-entropy when the target is 0 or 1 and the model uses a sigmoid probability. A confident wrong prediction receives a large penalty.

Categorical cross-entropy

Use categorical cross-entropy for one-of-many classification with softmax. The loss focuses on the probability given to the correct class.

Two target formats

Loss name	Target representation	Class 2 of 3
Categorical cross-entropy	One-hot vector	[0, 1, 0]
Sparse categorical cross-entropy	Integer class index	1 when indexing begins at 0

Compatibility map

Task	Output	Loss
Binary	Sigmoid	Binary cross-entropy
Multiclass	Softmax	Categorical cross-entropy
Multi-label	Sigmoid per label	Binary cross-entropy

Important

Some frameworks accept raw logits and apply the stable probability calculation inside the loss. Do not apply the activation twice.

16. Backpropagation

Finding responsibility for the mistake

Story step 5: Trace the error backward

Backpropagation starts from the loss and moves backward through the network. It calculates a gradient for every weight and bias. A gradient tells how the loss would change if that parameter changed slightly.

How this connects

After gradients are known, an optimizer decides the actual parameter update.

Chain rule in plain language

A weight affects a neuron. The neuron affects the next layer. The next layer affects the prediction. The prediction affects the loss. The chain rule connects all these effects.

CHAIN RULE

$$dL/dw = dL/d\text{prediction} * d\text{prediction}/dz * dz/dw$$

Gradient meaning

Gradient	Interpretation
Positive	Reducing the parameter may reduce loss
Negative	Increasing the parameter may reduce loss
Near zero	The parameter receives almost no update
Very large	The update may become unstable

Remember

Backpropagation calculates gradients. It does not choose the update rule. The optimizer uses those gradients.

17. Vanishing and Exploding Gradients

Why learning can become too weak or too unstable

Vanishing gradient

Backpropagation multiplies many derivatives. If many are smaller than 1, the final gradient reaching early layers can become extremely small. Those layers learn very slowly.

IN SYMBOLS

$\text{small} * \text{small} * \text{small} * \dots \rightarrow \text{almost zero}$

Exploding gradient

If repeated factors are large, gradients can become extremely large. Weight updates may jump wildly, and the loss can become unstable or undefined.

Symptoms and responses

Problem	What you may see	Common response
Vanishing	Early layers barely change	ReLU family, normalization, residual paths
Exploding	Loss spikes or becomes NaN	Lower learning rate, clipping, initialization
Dead ReLU	Many outputs stay zero	Leaky ReLU; review large updates

Learning-rate connection

Even correct gradients can produce poor learning if the learning rate is too high or too low. The learning rate controls the size of the optimizer step.

18. Optimizers and Gradient Descent

The rule that changes parameters

Story step 6: Correct the weights

The optimizer uses gradients to move parameters in a direction expected to reduce loss. Gradient descent is the basic update idea.

How this connects

Different optimizers change how gradients are averaged, scaled, or remembered, but all are trying to improve future predictions.

BASIC UPDATE

$$w_{\text{new}} = w_{\text{old}} - \text{learning_rate} * \text{gradient}$$

Learning rate

- Too large: updates can overshoot and training may oscillate or diverge.
- Too small: training can be very slow.
- A suitable value makes steady progress toward lower loss.

The hill analogy

Imagine standing on a foggy hill and trying to reach the lowest point. The gradient tells the steepest uphill direction, so moving in the opposite direction goes downhill. The learning rate is the step size.

Important limit of the analogy

Real neural-network loss surfaces have many dimensions, flat regions, curves, and saddle points. The hill story explains direction and step size, not every detail.

19. Batch, Stochastic, and Mini-Batch Learning

How many examples create one update?

Three choices

Method	Examples used per update	Easy interpretation
Batch gradient descent	Entire training set	Careful but expensive step
Stochastic gradient descent	One example	Fast, noisy step
Mini-batch gradient descent	Small group	Balanced and practical step

Epoch and iteration

An epoch is one complete pass through the training data. An iteration is one parameter update. With mini-batches, one epoch contains several iterations.

IN SYMBOLS

$$\text{iterations per epoch} = \text{ceil}(\text{dataset size} / \text{batch size})$$

Example

With 100,000 examples and a batch size of 1,000, one epoch has 100 iterations. Each iteration performs forward pass, loss calculation, backpropagation, and an optimizer update.

Why mini-batches are common

- Use memory more efficiently than a full-data update.
- Use vectorized hardware efficiently.
- Provide a useful amount of gradient noise without the extreme noise of one-example updates.

20. Momentum, Adagrad, RMSprop, and Adam

How optimizers improve the basic update

Momentum

Momentum remembers a moving average of recent gradients. It reduces side-to-side zig-zag movement and can speed progress in a consistent direction.

Adaptive optimizers

Optimizer	Main idea	Main caution
Adagrad	Shrinks each parameter step using accumulated squared gradients	Learning rate may become too small
RMSprop	Uses a moving average of squared gradients	Still requires sensible settings
Adam	Combines momentum-like averaging with RMSprop-like scaling	Strong default, but not always best

CONCEPT, NOT FULL FORMULA

Adam = first-moment average + squared-gradient average

Beginner starting strategy

- Start with Adam and a modest learning rate.
- Track training and validation loss.
- If learning is unstable, reduce the learning rate and inspect data and gradients.
- Compare another optimizer only after you have a reliable baseline.

Connection

The optimizer is one part of training. Good data, a suitable architecture, compatible activation and loss, and careful validation are equally important.

21. The Complete Story and Teaching Script

Use this page to explain deep learning to someone else

The story in nine sentences

- 1. We begin with examples containing input features and correct labels.
- 2. A neural network contains layers, and each layer contains neurons.
- 3. Every neuron combines inputs using learned weights and a bias.
- 4. An activation function transforms the weighted sum into a useful signal.
- 5. Forward propagation carries signals through the layers and produces a prediction.
- 6. A loss function measures the difference between the prediction and the correct answer.
- 7. Backpropagation uses the chain rule to calculate a gradient for every parameter.
- 8. An optimizer uses gradients and a learning rate to update the parameters.
- 9. Repeating this process across batches and epochs helps the model reduce loss and generalize to new data.

Final relationship map

Topic	Receives	Produces	Connects next to
Neuron	Features	Weighted signal	Activation
Activation	Weighted signal	Layer output	Forward propagation
Forward pass	Current parameters	Prediction	Loss
Loss	Prediction + target	Error value	Backpropagation
Backpropagation	Loss	Gradients	Optimizer
Optimizer	Gradients + learning rate	Updated parameters	Next forward pass

One-line definition to remember

Deep learning teaches a multi-layer neural network by repeating prediction, error measurement, gradient calculation, and parameter correction on many examples.

Source notes: Deep learning basics, activation functions, loss functions, and optimizers supplied by the user.

Beginner-friendly reference edition; formulas retained only where they support understanding.